PROJECT PROPOSAL

C-T-S — Connect Through Signs

Bachelor of Science in Computer Science (BsCS)

Project Title	CTS — Connect Through Signs
Project Summary	This project proposes **Connect Through Signs**, a real-time two-way American Sign Language (ASL) translation system. The system uses a webcam, OpenCV, MediaPipe, and machine learning to recognize hand gestures and convert them into text and optional speech. In the reverse direction, typed or spoken input is converted into corresponding signs and displayed through an animated 3D hand avatar. The prototype will support 1**0–20 commonly used signs** and will be developed as a React/FastAPI web application. The project aims to provide an accessible communication solution for everyday interactions between signing and non-signing individuals.
Expected Deliverables	SW (✓) HW ( ) Patent ( ) Paper (✓) Program Registration ( )
Advisor	Prof. Park Hakgwon
Expected Period	November 2026– March2027 (5-month duration, per the Section 3 timeline)

Major	Student ID	Year	Name	Contact Info
BsCS	232012021	4	Dua Naveed	dua.naveed.232012021@mypgi.edu.pk +92 335 9719960

Please ensure that both the email address and phone number are entered correctly and are reachable, as inconvenience may occur if you cannot be contacted.

Table of Contents

1. Project Background and Objectives 3

1.1 Definition of the Project

1.2 Technical Background of the Project

1.3 Problem Statement

1.4 Necessity of the Project

1.5 Final Goal to Be Achieved Through the Project

1.6 Detailed Goals to Achieve the Final Objective

2. Main Content of the Project 6

2.1 Requirements to Achieve the Final and Detailed Goals

2.2 Research on Fundamental Technologies

2.3 R&D Trends Aimed at Overcoming the Limitations of Current Technology

2.4 Specific Plans for the R&D to Be Actually Conducted

3. Implementation Plan 9

3.1 High-Level Monthly Plan

3.2 Monthly Plan at a Glance

4. Conclusion 11

4.1 Expectations for the Final Project Deliverables

4.2 Individual Capabilities Expected to Improve

4.3 Application Fields, and Academic and Industrial Expected Effects

4.4 Future Improvements Beyond This Project

5. References 13

Page numbers reflect the current layout; if content is edited, update them via References → Table of Contents → Update Table in Microsoft Word, or adjust manually.

1. Project Background and Objectives

1.1 Definition of the Project

This project proposes the development of a real-time, two-way sign language translation system that bridges communication between Deaf/hard-of-hearing individuals who use American Sign Language (ASL) and people who do not know sign language. ASL is a distinct, fully-fledged visual-gestural language with its own grammar and syntax rather than a manual encoding of English, and it is recognised as such by Deaf educational and cultural institutions [1], [2].

The system operates in two directions simultaneously:

1. **Sign → Text → Speech:** A webcam captures a user's hand gestures in real time, a machine learning model recognizes the signed content and converts it into written text, and that text can optionally be converted into spoken audio.
2. **Text → Sign → Avatar:** Typed or spoken input from the other party is matched against a sign vocabulary and rendered back as an animated 3D hand avatar performing the corresponding signs.

Unlike existing tools that address only one side of this exchange (e.g., captioning apps that convert speech to text, or ASL learning apps that teach isolated signs), this project is defined specifically as a two-way, real-time conversational bridge, scoped initially to a fixed vocabulary of common hand signs (hand shape and movement) to keep the system achievable within the project timeline.

1.2 Technical Background of the Project

The project draws on several established areas of computer science that have matured enough to be combined into a single working system:

- **Computer Vision / Hand Tracking:** Frameworks such as MediaPipe (built on top of OpenCV-based video pipelines) can extract precise hand landmark coordinates from a live camera feed in real time, without requiring custom-built pose-estimation models [3].
- **Sequence Modeling for Gesture Recognition:** Because signs are expressed through motion over time rather than a single static pose, recognition is framed as a sequence classification problem. Recurrent architectures such as LSTM (Long Short-Term Memory) and GRU (Gated Recurrent Unit) networks are well-suited to modeling this temporal structure [4], and are supported by a large body of prior sign-language-recognition research [5], [6].
- **Classical Machine Learning as a Baseline:** Alongside the deep learning approach, classical algorithms such as Random Forest and Support Vector Machines (SVM), implemented via Scikit-learn [7], provide a lightweight, fast-to-train baseline for simpler, more static signs, and serve as a benchmark against which the deep learning model's performance can be measured [8].
- **Text-to-Speech and Speech Synthesis:** Libraries such as pyttsx3 [9] allow spoken output to be generated entirely offline, without dependence on a paid cloud API or a live internet connection.
- **3D Avatar Rendering for the Reverse Direction:** The text-to-sign direction relies on 3D animation authored in Blender and exported in the glTF/GLB format [10] – the current standard for efficient, web-compatible 3D assets – which can then be rendered directly in a browser using Three.js [11].

- **Web Application Architecture:** The system is built as a web application (React on the frontend [12], FastAPI on the backend [13]) rather than a native mobile app, allowing it to run on any device with a camera and browser, and to be iterated on quickly without app-store review cycles.

Each of these components is individually well-established and production-tested; the technical contribution of this project lies in integrating them into one coherent, real-time, two-way pipeline.

1.3 Problem Statement

Deaf and hard-of-hearing individuals who communicate primarily through sign language face a recurring, structural communication barrier: most hearing people do not know sign language, and most everyday public spaces are not equipped to bridge that gap in real time.

This breaks down into a few concrete, specific problems:

- **No interpreter, most of the time.** Certified interpreters exist, but they must be booked in advance and are not present for spontaneous, everyday interactions – a spur-of-the-moment question, a walk-in visit, a quick transaction.
- **One-directional tools don't solve a two-way conversation.** Captioning apps convert speech to text, which helps a Deaf person understand a hearing speaker, but give the hearing person no way to understand signed responses. The reverse gap – turning sign into something a hearing person can understand – is left unaddressed.
- **Public-facing service interactions are a recurring pain point.** Ordering food at a counter, asking a retail employee a question, or checking in for an appointment all typically require either writing notes back and forth, relying on lip-reading, or the Deaf individual simply avoiding the interaction altogether.
- **The burden currently falls on the Deaf individual.** In the absence of a tool or interpreter, it is almost always the Deaf person who has to adapt – through writing, gesturing more broadly, or bringing a hearing companion – rather than the environment adapting to them.

SDG Alignment: This project supports SDG 10: Reduced Inequalities by providing a low-cost, real-time two-way ASL translation tool that helps Deaf individuals communicate independently in daily life.

1.4 Necessity of the Project

Solving the problem above is not a niche concern – it affects a large population, and the absence of a solution has a real, ongoing cost:

- The World Health Organization estimates that over 430 million people worldwide live with disabling hearing loss [14] – a substantial population for whom sign language is a primary means of communication, not a secondary translation of spoken language.
- The gap has persisted despite real technological progress elsewhere: hearing aids, live captioning, and video relay interpreting have all advanced significantly, yet almost none of them provide real-time, two-way communication in ordinary face-to-face settings – meaning the specific problem above has not actually closed as technology elsewhere has moved forward.
- Left unaddressed, this problem has a compounding social cost: it discourages independence, adds friction or exclusion to routine daily activities, and places the entire adaptive burden on the Deaf individual rather than on the environment around them.

- Because the underlying technical components (hand tracking, sequence modeling, offline speech synthesis, browser-based 3D rendering) are now mature and accessible, there is no longer a strong technical reason for this gap to persist – which is precisely what makes this project both necessary and achievable now.

1.5 Final Goal to Be Achieved Through the Project

The final goal of this project is to design, build, and demonstrate a working prototype of a real-time, two-way ASL-to-text/speech and text-to-ASL-avatar translation web application, capable of supporting a basic but genuine conversational exchange using a defined vocabulary of common signs – with a clear, demonstrable path toward deployment in real-world service environments such as cafés, restaurants, and shops, where it could help Deaf customers place orders and communicate with staff more comfortably and independently.

1.6 Detailed Goals to Achieve the Final Objective

To reach the final goal above, the project is broken down into the following detailed objectives:

3. **Build the sign-input pipeline:** Capture live video via webcam and OpenCV, extract hand landmarks in real time using MediaPipe [3], and structure the landmark data (via NumPy) for use as model input.
4. **Train and validate a sign-recognition model:** Train an LSTM/GRU sequence model on a fixed vocabulary of common signs, and train a Random Forest/SVM baseline (via Scikit-learn [7]) to benchmark recognition accuracy and inform which approach performs best for which sign types.
5. **Implement the sign-to-speech output stage:** Convert recognized signs into assembled text, and integrate optional offline text-to-speech (pyttsx3 [9]) so spoken output is available but never forced on the user.
6. **Build the reverse text-to-sign pipeline:** Implement vocabulary matching and a sign dictionary that maps incoming typed or spoken text to a corresponding sequence of known signs.
7. **Build and integrate the 3D avatar output:** Model and animate a hand/arm avatar in Blender, export it in glTF/GLB format [10], and render it in-browser via Three.js [11] so it can visually perform the mapped signs.
8. **Integrate both directions into a single web application:** Combine the sign-to-speech and text-to-sign pipelines into one React frontend backed by a FastAPI backend, so a full two-way conversation loop is possible in one interface.
9. **Test and validate the prototype:** Run end-to-end testing of both directions, and gather feedback from Deaf and/or interpreter reviewers to check accuracy, usability, and whether the tool genuinely eases communication rather than just technically functioning.
10. **Demonstrate a real-world use case:** Frame and present the prototype specifically around a service-counter scenario (e.g., café or restaurant ordering), to show a concrete, practical path from prototype to real deployment.

2. Main Content of the Project

2.1 Requirements to Achieve the Final and Detailed Goals

To reach the final goal and the detailed objectives set out in Section 1, the project has the following requirements, grouped by category:

Functional requirements

- Real-time capture and processing of webcam video, with hand landmark extraction fast enough to feel conversational (no perceptible lag between a sign being made and text appearing).
- Recognition accuracy sufficient to be usable for a defined, fixed vocabulary (50–100 common signs) rather than open, unconstrained vocabulary.
- A text output stage that assembles recognized signs into readable text, with speech output available as an optional toggle rather than a forced behavior.
- A reverse pipeline that accepts typed or spoken text, matches it against a sign dictionary, and drives a 3D avatar to perform the corresponding signs in a way a hearing person unfamiliar with sign language can still initiate, and a Deaf user can clearly follow.

Non-functional requirements

- The system must run as a browser-based web application, avoiding the need for app-store distribution or platform-specific native builds.
- Core processing should work with commodity hardware (a standard laptop webcam and browser), without requiring specialized capture equipment.
- All software components must be used in a way that respects their licenses, keeping the project reproducible and free of proprietary lock-in.
- The interface must be usable by both a Deaf/signing user and a non-signing user with no special training beyond normal app use.

Data requirements

- A labeled dataset of sign videos or landmark sequences covering the chosen fixed vocabulary, drawn from existing public sign-language datasets, supplemented with newly recorded samples where public coverage is thin.
- A defined sign dictionary and vocabulary list, agreed upon in advance, that both the recognition model and the avatar animation library are built around [15].
- Access to Deaf and/or interpreter feedback at multiple points in development, to validate that recognized/produced signs are actually correct and natural, not just technically classified correctly.

Environment and tooling requirements

- A consistent development environment (VS Code) across the frontend (React), backend (FastAPI), and machine learning components.
- Sufficient compute for model training (a standard CPU is workable for the classical baseline; a GPU-backed environment is preferable, though not strictly required, for training the LSTM/GRU model at reasonable speed).

- 3D content tooling (Blender) and a glTF/GLB export pipeline compatible with Three.js for the avatar output.

2.2 Research on Fundamental Technologies

Before committing to the architecture described in Section 1, the following areas were researched through published papers, technical documentation, and vendor/library resources:

- **Hand pose estimation and tracking:** Research into how modern hand-tracking systems (e.g., MediaPipe Hands) achieve real-time, on-device landmark detection without requiring a custom-trained pose model [3], which directly informed the decision to build on top of an existing, proven detector rather than training one from scratch.
- **Sign language recognition as a sequence problem:** Review of how prior sign-recognition research frames the task – treating a sign as a sequence of hand positions over time rather than a single image [4] – which supports the choice of LSTM/GRU sequence models over simple frame-by-frame image classifiers.
- **Public sign-language datasets:** Investigation of existing labeled datasets (such as WLASL, a word-level American Sign Language video dataset [5]) to understand what vocabulary coverage is realistically available without requiring the project to collect a large dataset entirely from scratch.
- **Classical machine learning baselines:** Review of when simpler models (Random Forest, SVM) remain competitive with deep learning for gesture classification, particularly for smaller datasets or simpler, more static signs [8], which supports using Scikit-learn-based models [7] as a benchmark rather than deep learning being the only option.
- **Offline text-to-speech engines:** Comparison of offline TTS libraries (such as pyttsx3 [9]) against cloud-based TTS APIs, weighing latency, cost, and internet-dependency trade-offs.
- **Web-based 3D rendering standards:** Study of the glTF/GLB format as the current standard for compact, efficient 3D assets on the web [10], and of Three.js as the dominant JavaScript library for rendering such assets directly in the browser without plugins [11].
- **Modern web application architecture:** Research into asynchronous backend frameworks (FastAPI [13]) and component-based frontend frameworks (React [12]) as they relate to handling real-time video/audio data streams alongside a responsive user interface.

2.3 R&D Trends Aimed at Overcoming the Limitations of Current Technology

Current sign-language-translation technology, across academic research and existing products, has several recognized limitations. Understanding these trends helped scope this project's approach and identify what is realistically achievable now versus what remains an open research problem:

- **From isolated signs to continuous signing:** Much existing work (and this project's initial scope) focuses on recognizing isolated, individually-performed signs. A visible research trend is moving toward continuous sign language recognition, where sentence-level grammar and sign transitions are modeled directly [6], [16] – an important direction for future iterations of this project, though out of scope for the current prototype.
- **From RNNs toward Transformer-based models:** While LSTM/GRU architectures remain effective and well-documented for sequence modeling, recent research trends show a shift toward Transformer-based architectures for sign language recognition and translation, offering better handling of long-

range temporal dependencies [6]. This project uses LSTM/GRU for feasibility within the project timeline, while acknowledging Transformer-based approaches as a natural future upgrade path.

- **Incorporating facial expression and non-manual markers:** A known limitation of hand-only recognition systems (including this project's planned scope) is that sign language grammar is also carried by facial expression and body posture [2]. Current research increasingly incorporates holistic, multi-modal tracking (hands, face, and pose together) to close this gap – a direction this project deliberately defers to keep the initial system achievable.
- **Data scarcity and synthetic augmentation:** A persistent limitation across sign-language ML research is the relative scarcity of large, diverse, well-labeled datasets compared to spoken-language NLP [5]. Trends addressing this include data augmentation techniques and synthetic sign generation to expand effective training data without requiring proportionally more manual recording and labeling.
- **From fixed animation clips toward generative sign production:** Many existing text-to-sign systems (including this project's planned approach) rely on a fixed library of pre-authored animation clips per word. An active research direction is generative sign production – synthesizing novel, natural sign motion directly from text rather than stitching together fixed clips [16] – which would allow smoother, less robotic avatar output in future versions.
- **On-device and real-time optimization:** There is a broader trend toward optimizing ML models (through quantization, lighter architectures, etc.) so that real-time inference can run reliably on standard consumer hardware rather than requiring specialized or cloud-based compute – directly relevant to this project's goal of running smoothly on an ordinary laptop webcam and browser.

2.4 Specific Plans for the R&D to Be Actually Conducted

Translating the above research and trends into a concrete, achievable plan for this project, the R&D work will proceed as follows:

1. **Dataset curation and preprocessing:** Select a fixed vocabulary (10–20 signs), assemble corresponding video/landmark data from public datasets (e.g., WLASL [5]) supplemented by newly recorded samples, and preprocess all samples into consistent MediaPipe landmark sequences using OpenCV for frame handling and NumPy for array structuring.
2. **Model development and comparison: Separately** Train LSTM/GRU sequence models on the curated dataset, and train a Random Forest/SVM baseline via Scikit-learn [7] on the same data. Compare accuracy, training time, and inference speed between the two, and use whichever performs best per sign category (or a combination) in the final pipeline – rather than assuming the deep learning model is automatically superior.
3. **Sign-to-speech pipeline integration:** Connect the trained recognition model's output to a text-assembly stage, then integrate pyttsx3 [9] for optional offline speech synthesis, and test end-to-end latency from a sign being performed to text (and optionally speech) being produced.
4. **Text-to-sign pipeline development:** Build the vocabulary-matching logic and sign dictionary, define an animation-ID mapping scheme, and author corresponding hand/arm avatar animations in Blender, exported as glTF/GLB assets [10].
5. **Avatar rendering integration:** Integrate the exported avatar assets into a Three.js-based renderer [11] within the React frontend, and connect it to the text-to-sign matching logic so that incoming text reliably triggers the correct animation sequence.

6. **Full system integration:** Combine both pipelines into a single React (frontend) and FastAPI (backend) web application, enabling a continuous two-way interaction loop within one interface.
7. **Testing and evaluation:** Conduct end-to-end testing covering recognition accuracy, system latency, and speech/avatar correctness, and gather structured feedback from Deaf and/or interpreter reviewers to validate real-world usability, not just technical function.
8. **Documentation of future research direction:** Record where this prototype's scope stops short of current R&D trends (e.g., continuous signing, facial expression, generative avatar motion) so that future work has a clear, explicit path forward rather than these limitations being discovered incidentally.

3. Implementation Plan

3.1 High-Level Monthly Plan

The project is planned across five months total, split into two phases: a 2-month prototype phase, proving the core sign-recognition and translation concept on a small vocabulary, followed by a 3-month expansion phase, which scales the vocabulary, completes the full bidirectional system, and delivers a tested, working web application.

Phase 1 – Prototype (Month 1 – Month 2)

Month 1: Core Recognition Foundation

- Set up the development environment (VS Code, React frontend scaffold, FastAPI backend scaffold).
- Implement real-time camera input capture using OpenCV.
- Integrate MediaPipe for hand landmark detection on the live video feed.
- Select an initial vocabulary of 5–8 ASL signs to prove the concept end-to-end [15].
- Collect/prepare landmark-sequence training data for the selected signs.
- Begin training an initial LSTM/GRU recognition model on this small vocabulary.

Month 2: Working Prototype

- Finish training and tuning the LSTM/GRU model for the 5–8 sign vocabulary; validate recognition accuracy.
- Connect the recognition output to a basic text-display stage.
- Demonstrate real-time sign → text recognition running end-to-end in the browser (React + FastAPI), even with a minimal interface.
- Conduct internal testing to confirm the core recognition pipeline is technically sound before scaling up.

Milestone at end of Month 2: a working prototype that can recognize 5–8 selected ASL signs in real time from a live camera feed and display them as text.

Phase 2 – Expansion to Final Product (Month 3 – Month 5)

Month 3: Vocabulary Expansion & Model Optimization

- Expand the ASL vocabulary from 5–8 signs to a larger, defined set (targeting 50–100 signs).
- Retrain and optimize the LSTM/GRU model on the expanded vocabulary; train a Random Forest/SVM baseline (via Scikit-learn) for comparison and benchmarking.
- Improve hand-tracking robustness (handling more varied hand positions, lighting, and signing speed).
- Begin building the text-to-sign side: vocabulary matching logic and the sign dictionary structure.

Month 4: Full Bidirectional Flow

- Complete the sign → text → speech direction, integrating optional offline text-to-speech (pyttsx3).

- Build the text → sign direction: finalize animation-ID mapping, author avatar animations in Blender, export as glTF/GLB, and integrate rendering via Three.js.
- Unify both directions into a single, coherent web-based interface (React + FastAPI), so a user can move between signing and reading/hearing responses within one screen.
- Begin performance evaluation: measure recognition accuracy, end-to-end latency, and avatar rendering smoothness.

Month 5: Completion, Testing, and Delivery

- Finalize the complete web application, resolving integration issues found during performance evaluation.
- Conduct structured user testing, including feedback from Deaf and/or interpreter reviewers, to validate real-world usability rather than just technical correctness.
- Fix issues identified during testing and polish the interface and interaction flow.
- Prepare final documentation and a demonstration of the complete two-way system, including the café/restaurant-style use case discussed in Section 1.

Milestone at end of Month 5: a complete, tested, two-way ASL web application – supporting an expanded vocabulary, sign-to-text-to-speech, text-to-sign avatar output, and validated through real user testing.

3.2 Monthly Plan at a Glance

Month	Phase	Focus	Key Deliverable
1	Prototype	Camera input, hand landmarks, initial LSTM/GRU on 5–8 signs	Core recognition pipeline running
2	Prototype	Finish training, connect to text output, internal testing	Working prototype: 5–8 signs recognized in real time
3	Final Product	Expand vocabulary, optimize models, begin text→sign logic	Larger vocabulary + optimized models
4	Final Product	Full bidirectional flow, avatar integration, performance evaluation	Complete sign↔text↔speech↔avatar flow
5	Final Product	Testing, user feedback, polish, final delivery	Finished, tested web application

4. Conclusion

4.1 Expectations for the Final Project Deliverables

By the end of the five-month project period, the final deliverable will be a complete, working two-way ASL translation web application, consisting of:

- A real-time sign recognition system covering an expanded vocabulary (50–100 signs), converting live hand signs captured through a webcam into text.
- An optional speech output, so recognized text can be spoken aloud when the user chooses.
- A reverse text-to-sign channel, where typed or spoken input is converted into signed output through an animated 3D hand avatar.
- A unified web interface where both directions work together in a single, continuous interaction – allowing a genuine two-way exchange between a signing and a non-signing person.
- Documented results from structured user testing, including feedback from Deaf and/or interpreter reviewers, validating that the system is not just technically functional but genuinely usable.

This deliverable is intended as a working, demonstrable proof of concept – production-ready in the sense that it functions end-to-end and has been tested with real users, but scoped deliberately (fixed vocabulary, hand-only recognition, web rather than native app) to be achievable within the project timeline, with a clear path to further expansion afterward (see 4.4).

4.2 Individual Capabilities Expected to Improve

Executing this project is expected to build practical, cross-disciplinary skills, including:

- **Computer vision:** hands-on experience with real-time video processing, hand landmark detection, and preparing visual data for machine learning.
- **Machine learning and deep learning:** practical experience training, tuning, and comparing sequence models (LSTM/GRU) against classical baselines (Random Forest/SVM), and evaluating them on a real, non-trivial recognition task rather than a textbook dataset.
- **Full-stack web development:** building and connecting a React frontend with a FastAPI backend, and handling real-time data (video, audio, animation state) across that stack.
- **3D content and animation pipelines:** learning to model, rig, and animate in Blender, and to bring that content into a browser through the glTF/GLB and Three.js pipeline.
- **Accessibility-conscious design:** designing an interface that must work for two very different users (a signing user and a non-signing user) at once, which builds a deeper, practical understanding of inclusive design beyond theory.
- **Working with real stakeholders:** incorporating feedback from Deaf and/or interpreter reviewers requires learning to design for, and take feedback from, a community the developer may not personally belong to – a skill relevant well beyond this one project.
- **Project planning and execution:** managing a multi-month, multi-phase technical project with defined milestones, which builds general skills in scoping, sequencing, and delivering complex work.

4.3 Application Fields, and Academic and Industrial Expected Effects

Application fields. While the prototype is scoped narrowly, the underlying system is directly applicable to a wide range of everyday settings where a Deaf person interacts with a non-signing person: cafés and restaurants (placing orders), retail shops (asking about products), schools, colleges, and universities (everyday interactions with staff and peers), clinics and public service counters, and customer-service kiosks more generally.

Academic effects. The project contributes a concrete, integrated case study in combining several active research areas – real-time hand tracking, sequence-based gesture recognition, and browser-based generative avatar output – into a single working system, rather than treating them as separate research problems. It also produces a documented comparison between a deep learning approach (LSTM/GRU) and a classical ML baseline (Random Forest/SVM) on a shared, practical task, which can inform future coursework or research choices in similar accessibility-technology projects.

Industrial and social effects. Beyond its academic value, the project demonstrates a low-cost, open-source-based alternative to either doing nothing or requiring a booked human interpreter for basic, everyday interactions. If adopted in commercial or public settings, it has the potential to reduce a real, ongoing source of exclusion for Deaf individuals in daily life, at a fraction of the cost of specialized interpreting services, while also serving as a foundation other teams or companies could build on for related accessibility products.

4.4 Future Improvements Beyond This Project

The five-month deliverable is intentionally scoped to be achievable, which means several valuable improvements are deliberately left for future work rather than attempted now:

- **Supporting more sign languages:** Expanding beyond ASL to other sign languages (e.g., British Sign Language, Indian Sign Language), since sign languages are not universal and each has its own grammar and vocabulary.
- **Supporting more spoken/written languages:** Extending the text and speech output beyond English, so the tool is useful to non-English-speaking hearing users as well.
- **AI-based sign correction and feedback:** One of the most valuable planned improvements is an intelligent feedback layer that recognizes what the user actually signed versus what they likely meant to sign, and gently flags the difference – for example, telling the user "It looks like you signed X – did you mean Y?" This would make the tool more forgiving for new or imperfect signers, and would double as an informal ASL learning aid, not just a translation tool.
- **From web app to mobile app:** Turning the system into a dedicated mobile app is a major planned next step, making it portable and usable anywhere a phone camera is available, rather than requiring a laptop and webcam.
- **Broader, everyday deployment:** The long-term vision for this project is for it to be usable anywhere a quick, informal conversation happens – cafés, restaurants, shops, schools, colleges, and universities – so that Deaf students and their families are not forced to pay extra for specialized institutions or dedicated interpreting support just to handle ordinary daily communication and participate normally in shared spaces.

5. References

References are numbered in order of first citation in the text (in-text citations appear in square brackets, e.g. [1]). Formatted as Author, Title, Publisher, Publication/Access Date, per the required reference format.

[1] National Association of the Deaf. American Sign Language. National Association of the Deaf. n.d. https://nad.org/knowledge-hub/american-sign-language/

[2] Gallaudet University. American Sign Language: A Language Recognized. Gallaudet University. n.d. https://gallaudet.edu/museum/exhibits/history-through-deaf-eyes/awareness-access-and-change/american-sign-language-a-language-recognized/

[3] Google AI Edge. MediaPipe Hands. Google. n.d. https://developers.google.com/mediapipe

[4] Goodfellow, I., Bengio, Y., & Courville, A. Deep Learning. MIT Press. 2016.

[5] Li, D., Rodriguez, C., Yu, X., & Li, H. Word-level Deep Sign Language Recognition from Video: A New Large-Scale Dataset and Methods Comparison. IEEE/CVF Winter Conference on Applications of Computer Vision (WACV) / arXiv. 2020.

[6] Camgoz, N. C., Hadfield, S., Koller, O., Ney, H., & Bowden, R. Neural Sign Language Translation. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 2018.

[7] Pedregosa, F., et al. Scikit-learn: Machine Learning in Python – Documentation. Scikit-learn. n.d. https://scikit-learn.org/

[8] Géron, A. Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow. O'Reilly Media. 2019 (2nd ed.).

[9] pyttsx3 Contributors. pyttsx3 Documentation. Python Package Index (PyPI). n.d. https://pypi.org/project/pyttsx3/

[10] Khronos Group, 3D Formats Working Group. glTF 2.0 Specification. Khronos Group. 11 October 2021 (Version 2.0.1). https://registry.khronos.org/glTF/specs/2.0/glTF-2.0.html

[11] Three.js Authors. Three.js Documentation. Three.js. n.d. https://threejs.org/docs/

[12] Meta Open Source. React Documentation. React. n.d. https://react.dev/learn

[13] Ramírez, Sebastián. FastAPI Documentation. FastAPI. n.d. https://fastapi.tiangolo.com/

[14] World Health Organization. Deafness and Hearing Loss. World Health Organization. 3 March 2026. https://www.who.int/en/news-room/fact-sheets/detail/deafness-and-hearing-loss

[15] Vicars, William G. American Sign Language University (ASL University). Lifeprint.com. 8 January 1997. https://www.lifeprint.com/asl101/

[16] Papers With Code. Sign Language Recognition. Papers With Code. n.d. https://paperswithcode.com/task/sign-language-recognition/latest